

# Dossiê Mestre — Projeto "Hail to the King"

Sep 18, 2026 · @isaque carvalho silva

## 1. Visão Executiva

**Nome do projeto:** "Hail to the King"

**Problema que o projeto pretende resolver:** demonstrar, de forma prática e verificável, competência em engenharia de dados moderna (arquitetura Medallion sobre Databricks/Delta Lake), aplicada a um domínio de negócio real que o autor já conhece profissionalmente (recuperação de crédito), com dois componentes adicionais de diferenciação: uma decisão automatizada via IA (Jev) e uma prova formal de regra de negócio (Bend2).

**Objetivo principal:** construir um pipeline de dados ponta a ponta (Bronze → Silver → Gold) tecnicamente correto e defensável em entrevista técnica, para um ambiente de dados de área de crédito.

**Contexto:** o autor é estagiário de TI/Data Science solo na Agecob, empresa de recuperação de crédito em Goiânia, e estudante de Data Science e IA na PUC Goiás (4º semestre). O projeto nasceu de um guia prático ("Building Azure Databricks and Medallion Experience for Free") sobre como construir experiência real em Databricks sem dados corporativos, usando as edições gratuitas disponíveis em 2026 (Databricks Free Edition + Azure for Students).

**Domínio de negócio:** recuperação de crédito / cobrança — o mesmo domínio da Agecob, escolhido deliberadamente para que o autor possa defender cada decisão de modelagem com conhecimento de negócio real.

**Tecnologias principais:**

- Databricks Free Edition (serverless) — PySpark, Spark SQL, Delta Lake, Unity Catalog
- Jev (TypeSafe AI) — modelo "System 1", usado via API para decisão estruturada
- Bend2 — linguagem de prova formal, usada isoladamente para verificar uma regra de negócio

**Arquitetura geral:** arquitetura Medallion (Bronze/Silver/Gold) rodando no Databricks, com o Jev integrado como um step de decisão estruturada entre Silver e Gold, e o Bend2 como peça isolada de verificação formal, conectada apenas conceitualmente ao domínio do pipeline (não integrada ao runtime).

**Resultado final esperado:** um repositório de portfólio público, documentado, com pipeline funcional, indicadores de negócio calculáveis, integração de IA funcionando, e uma regra de negócio formalmente provada — tudo defensável em entrevista técnica.

**Por que este projeto existe:** o autor aprende melhor praticando (cerca de 80% do aprendizado por aplicação) e precisa de experiência prática demonstrável em big data / Databricks para portfólio.

**Competências que o projeto pretende demonstrar:**

- Arquitetura de dados (Medallion, modelagem dimensional, SCD Type 2)
- Engenharia de dados aplicada com PySpark/Spark SQL/Delta Lake
- Ciência de dados como consumidora da camada Gold (EDA, modelo simples de propensão)
- Capacidade de integrar um modelo de IA de forma criteriosa (Jev), sabendo justificar a escolha
- Raciocínio formal/matemático sobre regras de negócio (Bend2)
- Disciplina de documentação técnica e de decisões arquiteturais

## 2. Contexto e Motivação

**Qual problema de negócio está sendo modelado:** a operação de uma área de crédito/cobrança — gerenciar uma carteira de dívidas, acordos de parcelamento, tentativas de contato e pagamentos, com o objetivo de maximizar recuperação e priorizar quem contatar primeiro.

**Por que recuperação de crédito foi escolhida como domínio:** é o domínio real de trabalho do autor na Agecob. Usar esse domínio permite defender cada decisão de modelagem com conhecimento de negócio genuíno, em vez de replicar um tutorial genérico de e-commerce.

**Relação com engenharia de dados:** o núcleo do projeto é um pipeline Medallion (Bronze/Silver/Gold) que transforma dados brutos e sujos de cobrança em informação confiável e modelada dimensionalmente.

**Relação com IA:** o Jev entra como um componente de decisão estruturada (não generativa) dentro do pipeline — classificando/pontuando dívidas quanto a propensão de pagamento ou prioridade de contato, na fronteira entre Silver e Gold.

**Relação com verificação formal:** o Bend2 é usado para provar formalmente que uma regra de negócio crítica (ex.: cálculo de parcelamento) nunca gera um estado inconsistente — uma garantia mais forte do que um teste unitário comum.

**Narrativa que conecta Databricks + Jev + Bend2:** os três compõem uma história de "arquitetura de dados tradicional + decisão estruturada de IA + verificação formal", mostrando que o autor sabe quando e onde aplicar cada tipo de ferramenta — não apenas empilhar tecnologias por estarem na moda.

**Papéis diferentes de cada tecnologia (importância explícita):**

- **Databricks/PySpark/Delta Lake** — motor de execução do pipeline de dados. É onde 100% do processamento pesado acontece.
- **Jev** — componente de decisão **integrado** ao pipeline, chamado via API como um step entre Silver e Gold.
- **Bend2** — componente **isolado**, de verificação formal, documentado à parte e conectado apenas conceitualmente ao domínio (não roda dentro do Spark/Databricks).

## 3. Objetivos

### 3.1 Objetivos técnicos

**Objetivo:** construir um pipeline de dados ponta a ponta (Bronze→Silver→Gold) tecnicamente correto, seguindo os padrões de arquitetura Medallion usados em produção.

| Objetivo                       | Por que existe                                                                       | Como será implementado                                       | Indicador de sucesso                       | Como verificar                                   |
|--------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------|--------------------------------------------------|
| Ingestão idempotente na Bronze | Rodar o job 2x não pode duplicar dado — padrão esperado de engenharia de dados séria | Merge/upsert ou checagem de chave na ingestão Delta          | Bronze idempotente                         | Rodar a ingestão 2x e contar linhas antes/depois |
| Qualidade de dados na Silver   | Dado bruto tem erros, duplicatas, nulos                                              | Pelo menos 1 regra de dedupe + 1 de validação de schema/nulo | Regra de qualidade aplicada e documentável | Inspecionar linhas rejeitadas/corrigidas         |

| Objetivo                            | Por que existe                                                   | Como será implementado                                         | Indicador de sucesso                                 | Como verificar                                                        |
|-------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------------------|
| SCD Type 2 em pelo menos 1 dimensão | Demonstrar histórico de mudanças (padrão clássico de engenharia) | Implementado na dimensão de cliente ou de acordo               | SCD2 funcionando                                     | Alterar um registro fonte e verificar histórico preservado            |
| Modelo dimensional real na Gold     | Mostrar que sabe modelar, não só empilhar tabela larga           | 1 fato + 2-3 dimensões                                         | Modelo dimensional presente                          | Revisar o schema da Gold                                              |
| Incremental load                    | Não fazer sempre full reload — padrão de produção                | Watermark ou Delta MERGE incremental                           | Incremental funcionando                              | Rodar com novo lote de dados e conferir que só o delta foi processado |
| Integração do Jev via API           | Mostrar decisão estruturada de IA integrada a um pipeline real   | Chamada HTTP com output tipado, plugada entre Silver e Gold    | Chamada funcionando com output estruturado           | Testar chamada e validar schema da resposta                           |
| Explicar a escolha do Jev           | Defender a decisão em entrevista                                 | —                                                              | Explicação de 1 minuto sobre "System 1 model" vs LLM | Explicar em voz alta sem consultar notas                              |
| Prova formal em Bend2               | Mostrar raciocínio formal sobre regra de negócio                 | <code>type / def / law</code> provando 1 regra real do domínio | Prova rodando com sucesso                            | Executar a prova e obter resultado válido                             |
| Explicar decisões de arquitetura    | Defensabilidade em entrevista                                    | —                                                              | Explicar cada decisão sem olhar código               | Simulação de entrevista                                               |

### 3.2 Objetivos de negócio

**Objetivo:** desenvolver um ambiente de dados para a área de crédito que entregue visibilidade e sinais de priorização sobre a carteira de cobrança.

| Objetivo                               | Problema que representa                            | Dado necessário                   | Cálculo/métrica                                  | Interpretação                       | Como será apresentado                                                                                |
|----------------------------------------|----------------------------------------------------|-----------------------------------|--------------------------------------------------|-------------------------------------|------------------------------------------------------------------------------------------------------|
| Taxa de recuperação visível            | Quanto da dívida total é efetivamente recuperado   | Valor da dívida, pagamentos       | % pago / total devido, por período               | Eficiência geral da cobrança        | Tabela/gráfico na Gold                                                                               |
| Aging da carteira mapeado              | Quanto da carteira está atrasada e há quanto tempo | Data de vencimento, data atual    | Distribuição por faixa (0-30, 31-60, 61-90, 90+) | Saúde da carteira                   | Gráfico de distribuição                                                                              |
| Taxa de adesão a acordos               | Quantos devedores aceitam negociar                 | Dívidas com/sem acordo            | % com acordo / total em cobrança                 | Eficácia da abordagem de negociação | Indicador simples                                                                                    |
| Taxa de quebra de acordo               | Quantos acordos firmados falham                    | Status do acordo (ativo/quebrado) | % quebrados / total de acordos firmados          | Qualidade dos acordos oferecidos    | Indicador simples                                                                                    |
| Score de propensão a pagamento         | Quem priorizar no contato                          | Histórico do devedor (via Jev)    | Score categórico/numérico por cliente/dívida     | Priorização de fila de cobrança     | Coluna na Gold, calculada pelo Jev                                                                   |
| Custo-efetividade por canal de contato | Qual canal (ligação, SMS, e-mail) converte mais    | Histórico de contato + resultado  | Taxa de conversão por canal                      | Onde investir esforço de cobrança   | Depende do dataset incluir histórico de contato — <b>NÃO DEFINIDO</b> se o schema final incluir isso |

### Sugestões adicionais — NÃO SÃO REQUISITOS ATUAIS:

- Segmentação de carteira via clustering (K-means) foi sugerida como exercício de ciência de dados, mas não foi confirmada como requisito do projeto.
- Modelo baseline de regressão logística para propensão de pagamento foi sugerido como comparação ao Jev, mas não foi confirmado como requisito.

## 4. Indicadores de Sucesso

### Indicadores técnicos

| Categoria | Indicador                          | Critério de sucesso                   | Como verificar                               | Status   |
|-----------|------------------------------------|---------------------------------------|----------------------------------------------|----------|
| Técnico   | Bronze idempotente                 | Rodar 2x não duplicado                | Rodar ingestão 2x, contar linhas             | Pendente |
| Técnico   | Qualidade de dados na Silver       | ≥1 regra de dedupe/validação aplicada | Inspecionar dados tratados                   | Pendente |
| Técnico   | SCD Type 2                         | Implementado em ≥1 dimensão           | Alterar registro fonte, checar histórico     | Pendente |
| Técnico   | Modelo dimensional na Gold         | 1 fato + 2-3 dimensões                | Revisar schema                               | Pendente |
| Técnico   | Incremental load                   | Não é sempre full reload              | Rodar com novo lote, checar delta processado | Pendente |
| Técnico   | Capacidade de explicar arquitetura | Explicar sem olhar código             | Simulação de entrevista                      | Pendente |

### Indicadores de negócio

| Categoria | Indicador                           | Critério de sucesso                         | Como verificar           | Status   |
|-----------|-------------------------------------|---------------------------------------------|--------------------------|----------|
| Negócio   | Taxa de recuperação calculável      | Métrica disponível por período na Gold      | Consultar tabela/gráfico | Pendente |
| Negócio   | Aging da carteira mapeado           | Distribuição por faixa de atraso disponível | Consultar gráfico        | Pendente |
| Negócio   | Taxa de adesão a acordos calculável | % disponível na Gold                        | Consultar indicador      | Pendente |

| Categoria | Indicador                           | Critério de sucesso                            | Como verificar           | Status   |
|-----------|-------------------------------------|------------------------------------------------|--------------------------|----------|
| Negócio   | Taxa de quebra de acordo calculável | % disponível na Gold                           | Consultar indicador      | Pendente |
| Negócio   | Score de propensão disponível       | Score por cliente/dívida via Jev               | Consultar coluna na Gold | Pendente |
| Negócio   | Custo-efetividade por canal         | Depende do schema incluir histórico de contato | <b>NÃO DEFINIDO</b>      | Pendente |

## Indicadores de integração

| Categoria  | Indicador                       | Critério de sucesso                                      | Como verificar      | Status                                                                     |
|------------|---------------------------------|----------------------------------------------------------|---------------------|----------------------------------------------------------------------------|
| Integração | Chamada de API do Jev funcional | Output estruturado e tipado, integrado como step da Gold | Testar chamada real | Pendente — acesso confirmado pelo autor, teste mínimo ainda não registrado |
| Integração | Prova formal em Bend2 funcional | law provada com sucesso                                  | Executar prova      | Pendente                                                                   |

## Indicadores de documentação/apresentação

| Categoria    | Indicador                                  | Critério de sucesso                                        | Como verificar          | Status   |
|--------------|--------------------------------------------|------------------------------------------------------------|-------------------------|----------|
| Documentação | README explica cada decisão de arquitetura | Pessoa consegue entender o projeto sem a conversa original | Revisão do README       | Pendente |
| Documentação | Projeto defensável em entrevista           | Autor explica em voz alta sem travar                       | Simulação de entrevista | Pendente |

Estes critérios binários foram definidos explicitamente na conversa e devem ser preservados — não substituídos por métricas vagas.

## 5. Arquitetura do Sistema

**Origem dos dados:** dados sintéticos, gerados via Faker (Python), simulando o domínio de recuperação de crédito (clientes, dívidas, acordos, pagamentos, histórico de contato). Não há dado corporativo real da Agecob envolvido.

**Geração dos dados sintéticos:** biblioteca Faker + Pandas/loop Python, rodando fora do Databricks (ambiente local ou notebook), gerando arquivos que depois são ingeridos na Bronze.

**Bronze:** ingestão raw dos dados sintéticos em formato Delta, sem transformação de negócio, com ingestão idempotente.

**Silver:** limpeza, dedupe, validação de schema/nulos, e SCD Type 2 em pelo menos uma dimensão (ex.: histórico do cliente).

**Gold:** modelagem dimensional (1 fato + 2–3 dimensões), incluindo o resultado da decisão do Jev como coluna/atributo.

**Delta Lake:** formato de armazenamento usado em todas as camadas (Bronze/Silver/Gold), dentro do Unity Catalog ( `hail_to_the_king.bronze` , `.silver` , `.gold` ).

**PySpark / Spark SQL:** motor de transformação usado nas 3 camadas, rodando em compute serverless do Databricks Free Edition.

**Databricks:** plataforma que hospeda todo o pipeline (workspace, compute serverless, Unity Catalog).

**Jev:** chamado via API HTTP como um step de decisão estruturada, consumindo dados da Silver e produzindo um output tipado (categoria/score) que é incorporado à Gold.

**API do Jev:** acesso confirmado pelo autor (via [console.typesafe.ai / docs.typesafe.ai](https://console.typesafe.ai/docs/typesafe.ai)). Detalhes de endpoint, autenticação e formato exato de request/response ainda **NÃO DEFINIDOS** — a confirmar na Fase 0, seção 0.3 do template de setup.

**Bend2:** não faz parte do runtime do pipeline. É uma peça isolada, rodando localmente (via Bun), que prova formalmente uma regra de negócio do domínio de crédito. Documentada em mini-repositório à parte, linkado no README principal.

**Componentes externos:** Azure for Students foi cogitado para a camada de orquestração (Azure Data Factory) e ADLS Gen2, mas seu uso efetivo no projeto **ainda não foi confirmado como decisão final** — ver Seção 23 (Pontos em Aberto).

**Fluxo dos dados:** Faker → Bronze (raw) → Silver (limpo/validado/histórico) → Jev (decisão estruturada) → Gold (modelo dimensional + indicadores de negócio).

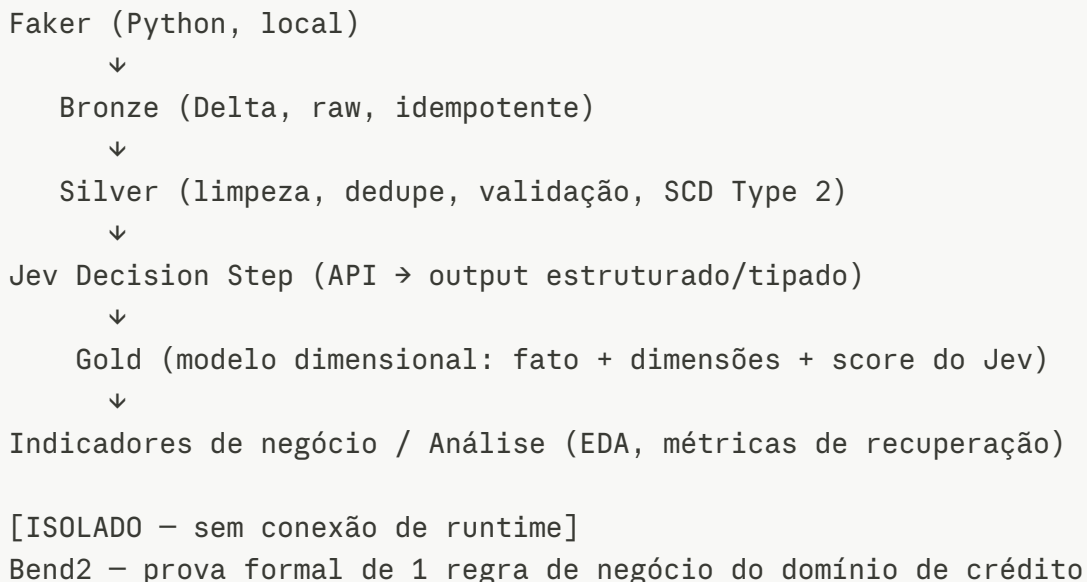
**Onde cada transformação acontece:** limpeza e qualidade → Silver; modelagem dimensional → Gold; decisão de IA → entre Silver e Gold (consome Silver, resultado vai para Gold).

**Onde o Jev entra:** como um step de pipeline, chamado a partir dos dados da Silver, antes da construção final da Gold.

**Onde o Bend2 entra:** fora do pipeline — como verificação formal isolada e documentada de uma regra de negócio (ver Seção 13).

**O que NÃO deve ser integrado:** Bend2 nunca deve rodar dentro do Spark/Databricks — essa integração foi explicitamente descartada (ver Seção 21 e 22): Bend2 compila para JS/C/binário nativo e não tem integração com Spark; tentar forçar isso consumiria tempo sem gerar valor de portfólio.

## Diagrama conceitual



## 6. Papel de Cada Tecnologia

| Tecnologia                | Papel no projeto                                                         | Por que foi escolhida                                                                                                                      | Onde é utilizada                   | O que preciso saber                                                                                           |
|---------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Databricks (Free Edition) | Plataforma que hospeda todo o pipeline                                   | Gratuita e perpétua; substitui a Community Edition (aposentada em 01/01/2026); tem Unity Catalog, Lakeflow, PySpark, Spark SQL, Delta Lake | Workspace, compute, catalog/schema | Serverless-only, com quotas (SQL warehouse 2X-Small, até 5 jobs concorrentes)                                 |
| PySpark                   | Motor de transformação dos dados                                         | Padrão de mercado para processamento distribuído                                                                                           | Bronze, Silver, Gold               | Transformações, DataFrames, operações incrementais                                                            |
| Spark SQL                 | Consultas e validações                                                   | Complementa PySpark para queries declarativas                                                                                              | Silver, Gold, análises             | Sintaxe SQL sobre DataFrames/tabelas Delta                                                                    |
| Delta Lake                | Formato de armazenamento das 3 camadas                                   | Suporta ACID, MERGE, time travel — essencial para idempotência e SCD2                                                                      | Bronze, Silver, Gold               | Comando MERGE, versionamento, schema evolution                                                                |
| Medallion Architecture    | Padrão arquitetural do pipeline                                          | Padrão reconhecido por recrutadores; estrutura clara de camadas                                                                            | Todo o pipeline                    | Bronze = raw, Silver = limpo, Gold = modelado para consumo                                                    |
| Faker                     | Geração de dados sintéticos                                              | Não há dado corporativo disponível; Faker gera volume rapidamente e casa com o domínio de crédito                                          | Antes da Bronze (geração)          | Geração de nomes, CPFs, datas, valores; limitações de escala (milhões, não dezenas de milhões sem otimização) |
| Jev                       | Componente de decisão estruturada (não generativa) integrado ao pipeline | "System 1 model", rápido (70-500ms), não alucina (output estruturado), acesso confirmado pelo autor                                        | Entre Silver e Gold                | É diferente de um LLM tradicional; produz output tipado, não texto livre                                      |

| Tecnologia    | Papel no projeto                                                | Por que foi escolhida                                                                                             | Onde é utilizada                                                 | O que preciso saber                                                         |
|---------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------|
| API do Jev    | Interface de chamada ao Jev                                     | É como o Jev é consumido no pipeline                                                                              | Step de decisão                                                  | Detalhes de endpoint/auth <b>NÃO DEFINIDOS</b> — a confirmar (Seção 12)     |
| Bend2         | Prova formal de 1 regra de negócio, isolada do pipeline         | Linguagem nova (dependent types + prova formal), mostra raciocínio matemático raro em portfólio júnior            | Mini-repositório à parte, linkado no README                      | Não roda dentro do Spark; compila para JS/C/binário via Bun                 |
| Azure         | Papel <b>ainda não confirmado</b> como decisão final do projeto | Foi cogitado no guia original para orquestração (ADF) e storage (ADLS Gen2), usando crédito do Azure for Students | Potencialmente camada de orquestração                            | <b>NÃO DEFINIDO</b> se será efetivamente usado neste projeto — ver Seção 23 |
| Unity Catalog | Organização lógica dos dados no Databricks                      | Recurso nativo da Free Edition, estrutura catalog/schema/table                                                    | Catalog<br>hail_to_the_king<br>com schemas<br>bronze/silver/gold | Namespace de 3 níveis:<br>catalog.schema.table                              |

Bend2 é tratado como peça isolada por decisão explícita da conversa — não faz parte da arquitetura de execução do pipeline.

## 7. Conceitos que Preciso Aprender

### Medallion Architecture (Bronze/Silver/Gold)

**O que é:** padrão de organização de dados em camadas progressivas de qualidade/refinamento. **Por que existe neste projeto:** é a espinha dorsal de todo o pipeline. **Onde aparece:** estrutura completa do Databricks (catalog hail\_to\_the\_king ). **O que preciso saber:** a função de cada camada e por que uma transformação pertence a uma e não a outra. **Pergunta de entrevista:** "Por que separar em 3 camadas em vez de fazer tudo direto?" **Resposta esperada:** separar responsabilidades permite reprocessar, auditar, isolar falhas e servir diferentes consumidores (raw para auditoria, limpo para BI, modelado para análise) sem acoplar tudo.

## Idempotência

**O que é:** propriedade de uma operação que produz o mesmo resultado independentemente de quantas vezes é executada. **Por que existe neste projeto:** requisito explícito da Bronze — rodar 2x não pode duplicar dado. **Onde aparece:** ingestão Bronze. **O que preciso saber:** como implementar via MERGE/upsert ou checagem de chave antes de inserir. **Pergunta de entrevista:** "Como você garante idempotência numa ingestão?" **Resposta esperada:** usando chave única + MERGE (upsert) em vez de INSERT simples, ou checando existência antes de gravar.

## Incremental load

**O que é:** processar apenas os dados novos/alterados, não o dataset inteiro a cada execução. **Por que existe neste projeto:** requisito técnico explícito, padrão de produção. **Onde aparece:** ingestão e transformações entre camadas. **O que preciso saber:** uso de watermark (coluna de data/timestamp) ou Delta MERGE incremental. **Pergunta de entrevista:** "Como você identifica o que é novo num carregamento incremental?" **Resposta esperada:** watermark (última data processada) comparado com timestamp dos novos registros, ou CDC.

## SCD Type 2

**O que é:** técnica de modelagem para preservar histórico de mudanças em uma dimensão (cria nova linha em vez de sobrescrever). **Por que existe neste projeto:** requisito técnico explícito. **Onde aparece:** Silver, em pelo menos 1 dimensão (ex.: histórico de cliente ou de acordo). **O que preciso saber:** colunas de controle (data\_inicio, data\_fim, flag\_atual), lógica de fechamento de linha antiga + inserção de nova. **Pergunta de entrevista:** "Qual a diferença entre SCD Type 1 e Type 2?" **Resposta esperada:** Type 1 sobrescreve o valor antigo (perde histórico); Type 2 cria uma nova linha versionada, preservando o histórico completo.

## Modelagem dimensional (fato e dimensão)

**O que é:** técnica de modelagem analítica com tabelas de fatos (eventos/métricas) e dimensões (contexto descritivo). **Por que existe neste projeto:** requisito da Gold. **Onde aparece:** Gold. **O que preciso saber:** distinguir o que é fato (ex.: pagamento, acordo) do que é dimensão (ex.: cliente, canal, tempo). **Pergunta de entrevista:** "Como você decidiu o que é fato e o que é dimensão no seu modelo?" **Resposta esperada:** fato = evento mensurável e granular (ex.: cada pagamento); dimensão = atributo descritivo usado para filtrar/agrupar (ex.: cliente, data, canal).

## Data quality / schema enforcement / schema evolution

**O que é:** garantias sobre a estrutura e integridade dos dados — enforcement impede dado fora do schema esperado, evolution permite o schema mudar de forma controlada. **Por que existe neste projeto:** requisito de qualidade na Silver. **Onde aparece:** Silver. **O que preciso saber:** como o Delta Lake trata schema enforcement por padrão e como habilitar evolution quando necessário. **Pergunta de entrevista:** "O que acontece se um dado chegar com schema diferente do esperado?" **Resposta esperada:** o Delta Lake rejeita por padrão (schema enforcement); é possível permitir evolução controlada via `mergeSchema`, mas deve ser uma decisão consciente, não automática.

## Deduplicação

**O que é:** remoção de registros duplicados. **Por que existe neste projeto:** requisito de qualidade na Silver. **Onde aparece:** Silver. **O que preciso saber:** definição de chave de negócio para identificar duplicatas, uso de window functions ou `dropDuplicates`. **Pergunta de entrevista:** "Como você identifica e trata duplicatas?" **Resposta esperada:** define-se uma chave de negócio (ou combinação de colunas) e usa-se `dropDuplicates` / window function para manter só o registro correto (ex.: mais recente).

## ETL/ELT

**O que é:** padrões de processamento de dados — Extract-Transform-Load vs Extract-Load-Transform. **Por que existe neste projeto:** contextualiza onde a transformação acontece (no Spark, após carregar na Bronze — característica de ELT). **Onde aparece:** todo o pipeline. **O que preciso saber:** diferença conceitual e por que Medallion se encaixa melhor em ELT. **Pergunta de entrevista:** "Esse pipeline é ETL ou ELT?" **Resposta esperada:** ELT — os dados são carregados raw na Bronze primeiro, e as transformações acontecem depois, dentro da plataforma (Silver/Gold).

## APIs e structured output

**O que é:** chamadas HTTP a um serviço externo (Jev), onde a resposta segue um schema definido em vez de texto livre. **Por que existe neste projeto:** é como o Jev é consumido. **Onde aparece:** step de decisão entre Silver e Gold. **O que preciso saber:** formato de request/response do Jev (**NÃO DEFINIDO** ainda — Seção 12), tratamento de erro de API. **Pergunta de entrevista:** "Por que usar structured output em vez de deixar o modelo responder livremente?" **Resposta esperada:** garante que a resposta seja sempre utilizável programaticamente, sem necessidade de parsing de texto livre nem risco de resposta inesperada.

## System 1 models (Jev) vs LLMs tradicionais

**O que é:** classe de modelo não-autoregressivo, focado em decisões rápidas e estruturadas, em contraste com LLMs generativos. **Por que existe neste projeto:** é a justificativa de escolha do Jev. **Onde aparece:** step de decisão do pipeline. **O que preciso saber:** Jev é "dois ordens de magnitude mais rápido" que LLMs (70-500ms), não é generativo, não alucina porque produz apenas outputs estruturados pré-definidos. **Pergunta de entrevista:** "Por que não usar um LLM comum para essa decisão?" **Resposta esperada:** LLMs generativos são mais lentos, mais caros e podem alucinar texto livre; um System 1 model como o Jev é otimizado para classificação/roteamento/scoring rápido com saída garantidamente tipada.

## Dependent types e provas formais (Bend2)

**O que é:** sistema de tipos onde tipos podem depender de valores, permitindo expressar e verificar propriedades matemáticas de um programa. **Por que existe neste projeto:** base teórica do Bend2, usada para provar uma regra de negócio. **Onde aparece:** mini-repositório isolado de verificação formal. **O que preciso saber:** a diferença entre testar (checar casos específicos) e provar (garantir para todos os casos possíveis). **Pergunta de entrevista:** "Qual a diferença entre um teste unitário e uma prova formal?" **Resposta esperada:** um teste unitário verifica casos específicos; uma prova formal garante matematicamente que a propriedade vale para todos os casos possíveis, dentro do sistema de tipos usado.

### `law` no Bend2

**O que é:** construção da linguagem Bend2 usada para declarar uma proposição (regra) que deve ser matematicamente verdadeira. **Por que existe neste projeto:** é o mecanismo usado para formalizar a regra de negócio escolhida. **Onde aparece:** arquivo `LAWS.bend` do mini-repositório Bend2. **O que preciso saber:** sintaxe básica de `type / def / law` e como o Bend2 verifica a prova computacionalmente. **Pergunta de entrevista:** "O que você provou com o Bend2 e como?" **Resposta esperada:** depende da regra escolhida (**NÃO DEFINIDO** ainda — Seção 13); estruturalmente, envolve declarar a regra como `law` e o Bend2 verificar que ela é válida para todo o domínio de entrada definido.

Esta seção é um mapa de conhecimento mínimo, não um curso completo. Conceitos de Big Data e arquitetura distribuída em geral (partições, shuffling) não foram discutidos em profundidade na conversa — **NÃO DEFINIDO** o nível de detalhe necessário; recomenda-se estudo complementar via Databricks Academy / Microsoft Learn (Seção 27).

## 8. Dataset Sintético

**Entidades conceituais discutidas:** clientes, dívidas, acordos, pagamentos, histórico de contato. Essas entidades foram mencionadas como o conteúdo esperado do dataset ("clientes, dívidas, acordos, pagamentos, histórico de contato"), mas **o schema formal (tabelas, colunas, tipos, chaves, relacionamentos) ainda não foi definido em detalhe** na conversa.

**Origem:** geração sintética via biblioteca Faker (Python), inspirada na recomendação do guia original ("gere um dataset sintético de recuperação de crédito — clientes, dívidas, acordos, pagamentos, histórico de contato").

**Volume:** o guia original sugeriu ~1 milhão de linhas geradas em minutos com Faker+Pandas; **este número não foi reconfirmado como decisão específica deste projeto** — tratar como referência, não como requisito fechado.

**Problemas artificiais para testar qualidade de dados:** mencionados de forma genérica (dedupe, nulos, validação de schema) mas **não especificados individualmente** — fica a critério da implementação da Silver decidir quais defeitos injetar propositalmente nos dados sintéticos para exercitar as regras de qualidade.

### Schema ainda não definido

Os seguintes pontos precisam ser decididos durante o desenvolvimento:

- Tabelas exatas (ex.: `clientes`, `dividas`, `acordos`, `pagamentos`, `contatos` ?) e seus nomes finais
- Colunas de cada tabela e seus tipos de dado
- Chaves primárias e estrangeiras
- Granularidade de cada tabela (ex.: 1 linha por dívida? 1 linha por parcela?)
- Se haverá tabela de histórico de contato (necessária para o indicador "custo-efetividade por canal", Seção 15)
- Volume final de linhas geradas

- Distribuições estatísticas usadas na geração (ex.: distribuição de valores de dívida, taxa de inadimplência simulada)
- Quais defeitos de qualidade serão injetados propositalmente (duplicatas, nulos, valores inconsistentes)

**NÃO DEFINIDO** — precisa ser decidido durante o desenvolvimento. Não inventar esses valores agora.

## 9. Bronze

**Finalidade:** ingestão raw dos dados sintéticos gerados via Faker, sem transformação de negócio.

**Estrutura:** schema `hail_to_the_king.bronze` no Unity Catalog (já criado no ambiente).

**Formato:** Delta Lake.

**Ingestão:** dos arquivos gerados pelo Faker para tabelas Delta na Bronze.

**Idempotência:** requisito explícito — rodar a ingestão 2x não pode duplicar dados. Mecanismo exato (MERGE por chave, checagem de existência) **NÃO DEFINIDO** ainda.

**Estratégia incremental:** requisito explícito — não deve ser sempre full reload. Mecanismo exato (watermark, CDC) **NÃO DEFINIDO** ainda.

**Tratamento de schema:** não discutido em detalhe; Delta Lake aplica schema enforcement por padrão.

**Armazenamento:** tabelas Delta dentro do catalog `hail_to_the_king`, schema `bronze`.

**Operações permitidas:** ingestão raw, adição de metadados técnicos (ex.: timestamp de ingestão), preservação do dado como recebido.

**Operações que NÃO devem acontecer nessa camada:** limpeza de dados, dedupe, validação de regra de negócio, modelagem dimensional — essas pertencem à Silver/Gold.

**Critérios de sucesso:** ver Seção 4 (indicadores técnicos) — ingestão idempotente é o critério binário definido.

## Fluxo conceitual

```

Arquivos Faker (CSV/Parquet)
    ↓
Leitura PySpark
    ↓
Escrita Delta em hail_to_the_king.bronze.<tabela>
(com checagem de idempotência via chave/MERGE)

```

## 10. Silver

**Limpeza:** tratamento de valores inconsistentes vindos da Bronze.

**Normalização:** padronização de formatos (datas, textos) — detalhes específicos **NÃO DEFINIDOS**.

**Dedução:** requisito explícito — pelo menos 1 regra de dedupe aplicada. Chave de negócio usada para identificar duplicatas **NÃO DEFINIDA** ainda.

**Validação:** requisito explícito — pelo menos 1 regra de validação de dado (schema ou nulo).

**Tratamento de nulos:** mencionado como uma das possíveis regras de qualidade; estratégia exata (descartar, imputar, marcar) **NÃO DEFINIDA**.

**Schema:** deve seguir schema enforcement do Delta; evolução controlada se necessário.

**Regras de qualidade:** pelo menos 1 de dedupe + 1 de validação de schema/nulo (critério explícito, Seção 4).

**SCD Type 2:** requisito explícito — implementado em pelo menos 1 dimensão. Qual dimensão exatamente (cliente? acordo?) **NÃO DEFINIDO**.

**Histórico:** preservado via SCD Type 2 na(s) dimensão(ões) escolhida(s).

**Chaves:** chave de negócio por entidade **NÃO DEFINIDA** ainda (depende do schema do dataset, Seção 8).

**Incrementalidade:** processamento incremental a partir da Bronze, consistente com o requisito geral de incremental load (Seção 4).

**Critérios de aceitação:** dedupe + validação + SCD2 funcionando (ver Seção 4).

## Por que cada transformação pertence à Silver e não à Bronze ou Gold

- **Não pertence à Bronze:** a Bronze deve preservar o dado exatamente como recebido, para fins de auditoria/reprocessamento. Limpeza e dedupe alterariam essa garantia.
- **Não pertence à Gold:** a Gold é uma camada de consumo, otimizada para análise e modelada dimensionalmente. Se a limpeza acontecesse ali, cada consumidor teria que lidar com inconsistências de novo, e o modelo dimensional ficaria acoplado a lógica de qualidade de dado.
- **Pertence à Silver:** é exatamente a camada intermediária responsável por transformar dado raw em dado confiável e consistente, antes de qualquer modelagem analítica.

## 11. Gold

**Modelo dimensional:** requisito explícito — 1 fato + 2–3 dimensões. Modelo exato **NÃO DEFINIDO** ainda.

**Fato (candidato conceitual, não confirmado):** provavelmente um fato de cobrança/pagamento (cada linha = um evento de pagamento ou o estado de uma dívida) — **HIPÓTESE / IDEIA** — não tratar como requisito.

**Dimensões (candidatas conceituais, não confirmadas):** cliente, tempo, canal de contato — **HIPÓTESE / IDEIA** — não tratar como requisito.

**Granularidade:** **NÃO DEFINIDA**.

**Métricas:** as métricas de negócio da Seção 15 (taxa de recuperação, aging, adesão a acordo, quebra de acordo, propensão) devem ser calculáveis a partir da Gold.

**Relacionamentos:** entre fato e dimensões, via chaves substitutas (surrogate keys) — padrão de modelagem dimensional; detalhes **NÃO DEFINIDOS**.

**Indicadores de negócio:** ver Seção 15 — todos devem ser calculáveis a partir do modelo da Gold.

**Dados provenientes do Jev:** o score/categoria de propensão a pagamento, produzido pelo step de decisão do Jev, deve estar presente na Gold (como atributo do fato ou de uma dimensão de cliente/dívida) — mecanismo exato de incorporação **NÃO DEFINIDO**.

**Consultas esperadas:** consultas analíticas para os indicadores de negócio (agregações por período, por faixa de atraso, por canal).

**Camada de consumo:** a Gold é a camada final, destinada a análise/EDA e aos indicadores de negócio; também mencionada como possível fonte para um dashboard futuro (não especificado como requisito deste projeto).

**O que falta:** modelo dimensional completo (nomes de tabelas fato/dimensão, colunas, chaves, granularidade) ainda precisa ser desenhado durante o desenvolvimento.

## 12. Jev

**O que é o Jev conforme a conversa:** primeiro modelo público da TypeSafe AI, da classe "System 1 Models". Diferente de chatbots/LLMs tradicionais, foi projetado para tomar decisões estruturadas rápidas que o software pode consumir diretamente. É descrito como "dois ordens de magnitude mais rápido e eficiente" que LLMs existentes, respondendo em 70–500ms. Não é generativo em texto livre — trabalha exclusivamente com saídas estruturadas, o que a documentação da TypeSafe descreve como eliminação do risco de alucinação.

**Por que foi escolhido:** o autor pediu explicitamente para incluir Jev e Bend2 no projeto por serem tecnologias novas que ele quer aprender. O papel do Jev foi desenhado para fazer sentido arquitetural (não só "porque é novo") — como componente de decisão estruturada dentro do pipeline.

**Qual problema ele resolve:** classificação/roteamento, extração de informação estruturada, scoring com base em probabilidades, decisões em tempo real de baixa latência — no caso deste projeto, aplicado a classificar dívidas por categoria de risco/propensão de pagamento.

**Por que não é simplesmente substituído por um LLM tradicional:** LLMs generativos são mais lentos, mais caros para esse tipo de tarefa, e podem produzir saída inesperada/alucinada em texto livre. O Jev garante schema fixo e velocidade.

**Como será integrado:** via chamada HTTP à API do Jev, consumindo dados da camada Silver e produzindo um output tipado que é incorporado à Gold.

**Onde entra no pipeline:** step de decisão entre Silver e Gold (ver Seção 5).

**Input:** dados de uma dívida/cliente vindos da Silver — estrutura exata **NÃO DEFINIDA**.

**Output:** categoria de risco/propensão de pagamento (ex.: `{categoria: "alta_prioridade" | "media" | "baixa", propensao_pagamento: float}`) — **HIPÓTESE / IDEIA** — não tratar como requisito, foi um exemplo ilustrativo dado na conversa, não um schema confirmado com a API real.

**Schema esperado:** structured output/JSON tipado — formato exato **A CONFIRMAR NA DOCUMENTAÇÃO DO JEV**.

**Tipo de decisão:** classificação/scoring (não generativa).

**Tratamento de erros:** **NÃO DEFINIDO**.

**Limitações:** **A CONFIRMAR NA DOCUMENTAÇÃO DO JEV** (rate limits, disponibilidade).

**Dependências:** acesso à API do Jev (confirmado pelo autor — ele mencionou ter acesso, após pesquisa sobre TypeSafe AI).

**Acesso/API:** console em `console.typesafe.ai`, documentação em `docs.typesafe.ai`. Pricing mencionado nas pesquisas: US\$0,042 por bilhão de tokens de entrada, tokens de saída gratuitos — **A CONFIRMAR NA DOCUMENTAÇÃO OFICIAL DO JEV**, já que é lançamento muito recente (setembro/2026) e pode ter mudado.

**O que ainda precisa ser confirmado na documentação oficial (checklist da Fase 0, seção 0.3 do template de setup):**

- Endpoint exato
- Método de autenticação
- Formato exato da requisição
- Formato exato da resposta
- Schema de structured output suportado
- Rate limits documentados
- Teste mínimo de chamada real (ainda não registrado nesta conversa)

**Justificativa arquitetural para o Jev como componente de decisão:** ele resolve um problema real (priorizar quem contatar na carteira de cobrança) usando uma classe de modelo adequada à tarefa (decisão rápida, tipada, sem necessidade de geração de texto), evitando o overhead e o risco de um LLM generativo tradicional para essa finalidade específica.

## 13. Bend2

**O que é Bend2:** linguagem de programação da Higher Order Company (mesmo criador do Bend original/HVM), combinando dependent types, checked program laws (leis/proposições verificadas) e execução paralela em CPU/GPU. Usa `def` para funções, `type` para dados, `law` para proposições. Lançada em setembro de 2026. Roda `.bend` files via Bun, ou compila para JavaScript, C ou binários nativos.

**Qual é seu propósito:** permitir escrever programas e provas usando as mesmas definições, viabilizando verificação formal de propriedades — a proposta é escrever *specs* (tipos precisos do que se quer) e deixar o próprio Bend2/IA provar a partir disso.

**Por que ele NÃO será usado para executar o pipeline:** decisão explicitamente tomada na conversa. O pipeline precisa rodar no runtime do Databricks (JVM/Spark), e o Bend2 não tem integração com Spark — compila para JS/C/binário nativo, sem I/O, orquestração ou processamento de DataFrames distribuídos. Forçar essa integração consumiria tempo sem gerar valor de portfólio.

**Qual parte do projeto ele representa:** uma peça isolada de verificação formal, documentada em mini-repositório à parte (ex.: `regras-formais-bend2/`), linkada no README principal com uma frase do tipo: "a regra de cálculo de parcelamento usada conceitualmente no pipeline foi verificada formalmente aqui".

**Regra de negócio candidata:** foi sugerido, como exemplo ilustrativo, provar que "um plano de parcelamento nunca gera saldo negativo" ou que "a soma das parcelas sempre bate com o valor acordado" — **HIPÓTESE / IDEIA — não tratar como requisito confirmado**. A regra final **NÃO FOI DECIDIDA**.

**type** : usado para declarar os dados da regra (ex.: estrutura de um plano de parcelamento) — estrutura exata **NÃO DEFINIDA**.

**def** : usado para declarar as funções/cálculos envolvidos (ex.: função que calcula parcelas) — **NÃO DEFINIDA**.

**law** : usado para declarar a proposição que deve ser sempre verdadeira (ex.: soma das parcelas = valor total) — **NÃO DEFINIDA**.

**Prova formal:** o Bend2 verifica computacionalmente que a `law` é válida para todo o domínio de entrada — mecanismo específico a estudar durante a implementação.

**Estrutura esperada do mini-repositório:** inspirada no exemplo de jogo da documentação oficial do Bend2, que mantém transições de estado em `main.bend`, especificações em `LAWS.bend` e provas em `PROOF.bend`.

**Relação conceitual entre a regra do Bend2 e o domínio de crédito:** a regra escolhida deve vir do domínio de recuperação de crédito (ex.: cálculo de acordo/parcelamento), reforçando a narrativa unificada do projeto, mesmo estando isolada tecnicamente do pipeline.

**Bend2 NÃO faz parte do runtime do pipeline Databricks.** Ele é uma peça isolada de verificação formal conectada conceitualmente ao projeto — esta é uma decisão arquitetural definitiva (ver Seções 21 e 22).

## 14. Regras de Negócio

| Regra                                                                | Origem                                      | Importância                                 | Dados necessários                  | Onde é aplicada                        | Implementada no pipeline? | Proxada no Bend2?         | Status   | Tipo             |
|----------------------------------------------------------------------|---------------------------------------------|---------------------------------------------|------------------------------------|----------------------------------------|---------------------------|---------------------------|----------|------------------|
| Soma das parcelas de um acordo deve bater com o valor total acordado | Sugerida como exemplo de regra a formalizar | Garante consistência financeira dos acordos | Estrutura do plano de parcelamento | Domínio de negócio / camada de acordos | Não confirmado            | Candidata, não confirmada | Hipótese | Regra de negócio |
| Um plano de parcelamento nunca gera saldo negativo                   | Sugerida como exemplo de regra a formalizar | Evita inconsistência financeira             | Estrutura do plano de parcelamento | Domínio de negócio / camada de acordos | Não confirmado            | Candidata, não confirmada | Hipótese | Regra de negócio |
| Ingestão na Bronze deve ser idempotente                              | Requisito técnico explícito                 | Padrão de engenharia sério                  | Chave de negócio por registro      | Bronze                                 | Sim (requisito)           | Não                       | Definida | Regra técnica    |
| Pipeline deve suportar carga incremental                             | Requisito técnico explícito                 | Padrão de produção                          | Watermark/timestamp                | Bronze/Silver                          | Sim (requisito)           | Não                       | Definida | Regra técnica    |

### Diferenciação explícita:

- **Regra de negócio:** trata de lógica do domínio de crédito (ex.: consistência de parcelamento). A regra final que será provada no Bend2 **ainda não foi escolhida definitivamente**.
- **Regra técnica:** trata de qualidade/comportamento do pipeline (idempotência, incrementalidade) — já são requisitos confirmados, mas de engenharia, não de negócio.
- **Hipótese de modelagem:** as regras de negócio candidatas (parcelamento) foram citadas como exemplos ilustrativos na conversa, nunca como decisão final.

## 15. Métricas e Indicadores de Negócio

### Taxa de recuperação

1. **Nome:** Taxa de recuperação
2. **Definição:** % do valor total em dívida que foi efetivamente pago, por período
3. **Fórmula:**  $\text{soma(valor pago)} / \text{soma(valor devido)}$ , por período
4. **Granularidade:** por período (mensal? **NÃO DEFINIDO**)
5. **Campos necessários:** valor da dívida, valor pago, data
6. **Exemplo:** não calculado (dado ainda não gerado)
7. **Interpretação:** eficiência geral da cobrança
8. **Camada de cálculo:** Gold
9. **Apresentação:** tabela/gráfico

### Aging da carteira

1. **Nome:** Aging da carteira
2. **Definição:** distribuição de dívidas por faixa de atraso
3. **Fórmula:** contagem/soma de dívidas agrupadas em faixas (0-30, 31-60, 61-90, 90+ dias de atraso)
4. **Granularidade:** por faixa de atraso
5. **Campos necessários:** data de vencimento, data atual
6. **Exemplo:** não calculado
7. **Interpretação:** saúde da carteira
8. **Camada de cálculo:** Gold
9. **Apresentação:** gráfico de distribuição

### Taxa de adesão a acordos

1. **Nome:** Taxa de adesão a acordos
2. **Definição:** % de dívidas com acordo firmado vs. total em cobrança
3. **Fórmula:**  $\text{contagem(dividas com acordo)} / \text{contagem(total de dividas em cobranca)}$
4. **Granularidade:** total ou por período
5. **Campos necessários:** status da dívida (com/sem acordo)

6. **Exemplo:** não calculado
7. **Interpretação:** eficácia da abordagem de negociação
8. **Camada de cálculo:** Gold
9. **Apresentação:** indicador simples

### Taxa de quebra de acordo

1. **Nome:** Taxa de quebra de acordo
2. **Definição:** % de acordos que entraram em default depois de firmados
3. **Fórmula:**  $\text{contagem}(\text{acordos quebrados}) / \text{contagem}(\text{total de acordos firmados})$
4. **Granularidade:** total ou por período
5. **Campos necessários:** status do acordo (ativo/quebrado)
6. **Exemplo:** não calculado
7. **Interpretação:** qualidade dos acordos oferecidos
8. **Camada de cálculo:** Gold
9. **Apresentação:** indicador simples

### Score/propensão de pagamento

1. **Nome:** Score de propensão a pagamento
2. **Definição:** score por cliente/dívida indicando probabilidade/prioridade de pagamento
3. **Fórmula:** calculado pelo Jev (não é fórmula determinística definida pelo autor — é saída do modelo)
4. **Granularidade:** por cliente/dívida
5. **Campos necessários:** histórico do devedor (estrutura exata **NÃO DEFINIDA**)
6. **Exemplo:** não calculado
7. **Interpretação:** priorização de fila de cobrança
8. **Camada de cálculo:** Gold (via chamada ao Jev entre Silver e Gold)
9. **Apresentação:** coluna na Gold

### Custo-efetividade por canal de contato

1. **Nome:** Custo-efetividade por canal
2. **Definição:** taxa de conversão por canal de contato (ligação, SMS, e-mail)
3. **Fórmula:**  $\text{contagem}(\text{pagamentos apos contato}) / \text{contagem}(\text{contatos})$  , por canal

4. **Granularidade:** por canal

5. **Campos necessários:** histórico de contato + resultado — **depende do schema final incluir essa tabela (Seção 8, NÃO DEFINIDO)**

6. **Exemplo:** não calculado

7. **Interpretação:** onde investir esforço de cobrança

8. **Camada de cálculo:** Gold

9. **Apresentação:** indicador por canal — **condicional à existência do dado de contato**

Nenhum valor de resultado foi calculado ainda — os dados sintéticos ainda não foram gerados. Estas definições são a especificação do que deve ser calculado, não resultados reais.

## 16. Fluxo End-to-End

| # | Etapa            | Entrada          | Processamento                                     | Saída                              | Tecnologia        | Critério de sucesso                  |
|---|------------------|------------------|---------------------------------------------------|------------------------------------|-------------------|--------------------------------------|
| 1 | Gerar dados      | —                | Script Faker gera entidades sintéticas de crédito | Arquivos (CSV/Parquet)             | Python + Faker    | Dados gerados com schema consistente |
| 2 | Ingerir          | Arquivos gerados | Leitura via PySpark                               | DataFrame Spark                    | PySpark           | Leitura sem erro                     |
| 3 | Armazenar Bronze | DataFrame        | Escrita Delta idempotente                         | Tabelas Delta em <b>bronze</b>     | Delta Lake        | Rodar 2x não duplica                 |
| 4 | Validar/limpar   | Tabelas Bronze   | Validação de schema/nulos                         | Dados validados                    | PySpark/Spark SQL | Regra de validação aplicada          |
| 5 | Deduplicar       | Dados validados  | Remoção de duplicatas por chave                   | Dados únicos                       | PySpark           | Regra de dedupe aplicada             |
| 6 | Histórico (SCD2) | Dados únicos     | Versionamento de dimensão                         | Tabela Silver com histórico        | Delta MERGE       | SCD2 funcionando                     |
| 7 | Decisão do Jev   | Dados da Silver  | Chamada API com structured output                 | Score/categoria por cliente/dívida | API Jev           | Output estruturado válido            |

| #  | Etapa                | Entrada                   | Processamento                  | Saída                | Tecnologia        | Critério de sucesso          |
|----|----------------------|---------------------------|--------------------------------|----------------------|-------------------|------------------------------|
| 8  | Modelar Gold         | Silver + resultado Jev    | Modelagem dimensional          | Fato + dimensões     | PySpark/Spark SQL | Modelo dimensional presente  |
| 9  | Calcular indicadores | Gold                      | Agregações (Seção 15)          | Métricas de negócio  | Spark SQL         | Métricas calculáveis         |
| 10 | Executar prova Bend2 | Regra de negócio definida | type / def / law + verificação | Prova válida         | Bend2 (isolado)   | Prova executando com sucesso |
| 11 | Documentar           | Todo o projeto            | Escrita de README, diagramas   | Documentação         | Markdown          | README explica decisões      |
| 12 | Apresentar           | Projeto completo          | Preparação de pitch/entrevista | Narrativa defensável | —                 | Autor explica sem travar     |

Etapas 1–6 e 8–9 são sequenciais dentro do Databricks; etapa 7 (Jev) é uma chamada externa via API; etapa 10 (Bend2) roda em paralelo/isolada, sem dependência direta do restante do fluxo.

## 17. Estrutura do Repositório

### Estrutura definida

- Nome do repositório: **Hail to the King** (codinome do projeto, confirmado pelo autor)
- Catalog Databricks correspondente já criado: `hail_to_the_king` (Unity Catalog, tipo Standard, região us-east-2), com schemas `bronze`, `silver`, `gold`

## Estrutura recomendada (não confirmada como decisão final)

```
hail-to-the-king/
├── README.md
├── docs/           # documentação, diagramas, decision records
├── data/           # scripts/saídas do gerador Faker (dados
                    # sintéticos)
├── notebooks/      # notebooks Databricks (Bronze/Silver/Gold)
├── src/            # código PySpark reutilizável (funções de
                    # transformação)
├── tests/          # testes (pytest) de qualidade/transformação
├── databricks/     # Databricks Asset Bundles (databricks.yml),
                    # configs de job
├── jev/            # código de integração com a API do Jev
├── bend2/          # mini-repositório isolado: main.bend, LAWS.bend,
                    # PROOF.bend
```

Esta estrutura de diretórios é uma sugestão baseada em padrões observados nos repositórios de referência (Seção 27), **não uma decisão já tomada** pelo autor. Nomes de pastas podem mudar durante a implementação.

**Pendências de configuração do repositório** (do template de Fase 0, seção 0.6): branch principal, `.gitignore`, gerenciamento de secrets, organização de commits — **NÃO DEFINIDOS** ainda nesta conversa.

## 18. Plano de Implementação

Nenhuma estimativa de tempo é dada abaixo por fase, pois isso não foi discutido na conversa — apenas dependências e critérios de conclusão.

### Fase 0 — Setup

**Objetivo:** deixar todo o ambiente configurado, testado e documentado antes de qualquer implementação. **Tarefas:** workspace Databricks, compute, catalog/schema (concluído: `hail_to_the_king` com bronze/silver/gold), storage/lakehouse, notebooks/jobs, secrets, bibliotecas, Azure (se aplicável), Jev (teste de conexão real), Bend2 (instalação local), ambiente local, repositório Git, smoke tests. **Dependências:** nenhuma (é a fase inicial). **Resultado esperado:** ambiente 100% funcional e documentado. **Critério de conclusão:**

Definition of Done da Fase 0 (12 itens do template de setup) totalmente atendida.

**Conhecimentos necessários:** navegação básica no Databricks, conceitos de Unity Catalog. **Status atual:** parcialmente concluída — Workspace, Compute e Catalog/Schema fechados; Storage/Lakehouse, Notebooks/Jobs, Secrets, Bibliotecas, Azure, Jev (teste real), Bend2, Ambiente local, Repositório e Smoke tests ainda pendentes.

## Fase 1 — Dataset

**Objetivo:** gerar o dataset sintético de recuperação de crédito. **Tarefas:** definir schema final (Seção 8), escrever script Faker, gerar volume de dados. **Dependências:** Fase 0 concluída (ambiente local funcionando). **Resultado esperado:** arquivos de dados sintéticos prontos para ingestão. **Critério de conclusão:** dados gerados com schema consistente.

## Fase 2 — Bronze

**Objetivo:** implementar ingestão raw idempotente. **Tarefas:** ler arquivos Faker, escrever em Delta na Bronze, garantir idempotência. **Dependências:** Fase 1. **Resultado esperado:** tabelas Bronze populadas. **Critério de conclusão:** rodar 2x não duplica dados.

## Fase 3 — Silver

**Objetivo:** limpar, validar, deduplicar e implementar SCD Type 2. **Tarefas:** regras de qualidade, dedupe, SCD2 em 1 dimensão, incrementalidade. **Dependências:** Fase 2. **Resultado esperado:** tabelas Silver confiáveis e com histórico. **Critério de conclusão:** dedupe + validação + SCD2 funcionando.

## Fase 4 — Jev

**Objetivo:** integrar a decisão estruturada do Jev ao pipeline. **Tarefas:** confirmar API (Seção 12), implementar chamada, tratar erros, incorporar resultado. **Dependências:** Fase 3 (consome dados da Silver) + acesso à API confirmado na Fase 0. **Resultado esperado:** score/categoria por cliente/dívida disponível. **Critério de conclusão:** chamada funcionando com output estruturado válido.

## Fase 5 — Gold

**Objetivo:** modelar dimensionalmente e calcular indicadores. **Tarefas:** desenhar fato + dimensões, incorporar resultado do Jev, calcular métricas (Seção 15). **Dependências:** Fases 3 e 4. **Resultado esperado:** modelo dimensional consultado, indicadores de negócio

calculáveis. **Critério de conclusão:** modelo dimensional presente (1 fato + 2–3 dimensões) e métricas calculáveis.

## Fase 6 — Bend2

**Objetivo:** provar formalmente 1 regra de negócio. **Tarefas:** escolher a regra final (Seção 14), escrever `type / def / law`, executar prova. **Dependências:** ambiente Bend2 instalado (Fase 0); pode rodar em paralelo às Fases 1–5. **Resultado esperado:** prova formal funcionando, documentada em mini-repositório. **Critério de conclusão:** prova executa com sucesso.

## Fase 7 — Validação

**Objetivo:** testar todo o pipeline e suas garantias. **Tarefas:** ver Seção 20 (testes e validação). **Dependências:** Fases 2–6 concluídas. **Resultado esperado:** confiança de que todos os critérios binários (Seção 4) estão atendidos. **Critério de conclusão:** checklist de implementação (Seção 19) completo.

## Fase 8 — Documentação

**Objetivo:** escrever README e documentação de arquitetura. **Tarefas:** ver Seção 26 (como o projeto deve aparecer no portfólio). **Dependências:** Fases 1–7. **Resultado esperado:** README completo, diagramas, decisões documentadas. **Critério de conclusão:** alguém entende o projeto sem ter participado da conversa original.

## Fase 9 — Portfólio

**Objetivo:** publicar e preparar apresentação/entrevista. **Tarefas:** publicar repositório, preparar pitch (Seção 25). **Dependências:** Fase 8. **Resultado esperado:** projeto publicado e defensável. **Critério de conclusão:** autor consegue explicar o projeto de ponta a ponta sem travar.

# 19. Checklist de Implementação

## Fase 0 — Setup

- ✓ ~~Databricks workspace criado (Free Edition, região us-east-2)~~
- ✓ ~~Compute confirmado (serverless-only, SQL Warehouse 2X-Small, Spark 4.2.0)~~
- ✓ ~~Catalog `hail_to_the_king` criado (Standard)~~

- ☒ ~~Schemas bronze/silver/gold criados~~
- ☐ Storage/Lakehouse documentado em detalhe
- ☐ Notebooks e Jobs documentados
- ☐ Secret scope configurado (para API key do Jev)
- ☐ Bibliotecas e dependências listadas com versões
- ☐ Azure documentado (se aplicável — **NÃO DEFINIDO** se será usado)
- ☐ Acesso ao Jev confirmado e testado com chamada mínima real
- ☐ Bend2 instalado localmente e exemplo mínimo executado
- ☐ Ambiente local documentado (Python, Git, Bun, etc.)
- ☐ Repositório Git configurado (branch, .gitignore, secrets)
- ☐ Smoke tests completos

## Dataset e Pipeline

- ☐ Schema do dataset sintético definido
- ☐ Dataset gerado via Faker
- ☐ Bronze funcionando
- ☐ Bronze idempotente
- ☐ Silver funcionando
- ☐ Deduplicação implementada
- ☐ Tratamento de nulos implementado
- ☐ SCD Type 2 implementado
- ☐ Incremental load funcionando

## Jev e Gold

- ☐ API Jev funcionando
- ☐ Output estruturado válido
- ☐ Gold com modelo dimensional (fato + dimensões)
- ☐ Indicadores de negócio calculáveis (Seção 15)

## Bend2

- ☐ Regra de negócio escolhida (Seção 14)
- ☐ Regra formalizada em Bend2

- ☐ Prova Bend2 funcionando

## Fechamento

- ☐ Testes escritos (Seção 20)
- ☐ README completo
- ☐ Arquitetura documentada
- ☐ Projeto defensável em entrevista

## 20. Testes e Validação

| Tipo de teste                | O que prova                                                                                        | Status                  |
|------------------------------|----------------------------------------------------------------------------------------------------|-------------------------|
| Testes de ingestão           | A Bronze recebe os dados corretamente a partir dos arquivos gerados                                | Não definido em detalhe |
| Testes de qualidade          | As regras de dedupe/validação da Silver realmente eliminam os defeitos esperados                   | Não definido em detalhe |
| Testes de transformação      | As transformações Silver→Gold produzem o modelo dimensional esperado                               | Não definido em detalhe |
| Testes de incrementalidade   | Rodar com novo lote processa só o delta, não tudo de novo                                          | Não definido em detalhe |
| Testes de idempotência       | Rodar a ingestão 2x não duplica dados                                                              | Não definido em detalhe |
| Testes da integração com Jev | A chamada à API retorna output válido no schema esperado, e erros são tratados                     | Não definido em detalhe |
| Testes das métricas          | Os cálculos de indicadores de negócio (Seção 15) batem com valores esperados em um caso controlado | Não definido em detalhe |
| Testes/provas Bend2          | A <code>law</code> escolhida é verificada com sucesso pelo Bend2                                   | Não definido em detalhe |

| Tipo de teste     | O que prova                             | Status                  |
|-------------------|-----------------------------------------|-------------------------|
| Validações finais | Checklist completo da Seção 19 atendido | Não definido em detalhe |

O guia original mencionou o uso de **pytest** para testes unitários em um fluxo típico de CI/CD com Databricks Asset Bundles (dev→staging→produção), mas isso foi citado como referência de mercado, não como decisão confirmada para este projeto especificamente. A estrutura detalhada de cada teste **precisa ser decidida durante o desenvolvimento**.

## 21. Decisões Arquiteturais

### Decisão: usar Medallion Architecture

**Motivo:** padrão reconhecido por recrutadores de dados júnior; demonstra separação de responsabilidades entre raw, limpo e modelado. **Alternativas consideradas:** nenhuma alternativa foi discutida — Medallion foi o ponto de partida desde o guia original. **Status:** Definido.

### Decisão: usar Databricks (Free Edition)

**Motivo:** única opção gratuita e perpétua disponível em 2026 após a aposentadoria da Community Edition (01/01/2026); suporta PySpark, Delta Lake, Unity Catalog e pipelines Lakeflow. **Alternativas consideradas:** nenhuma alternativa de plataforma foi discutida. **Status:** Definido — já implementado (workspace, compute, catalog criados).

### Decisão: usar Jev via API

**Motivo:** demonstrar integração com IA de estado da arte de forma criteriosa; o autor confirmou ter acesso à API. **Alternativas consideradas:** usar um LLM tradicional para a mesma decisão — descartado por ser mais lento, mais caro e sujeito a alucinação para uma tarefa de classificação/scoring estruturado. **Por que não foi escolhido (LLM tradicional):** não oferece garantia de schema fixo nem a velocidade do Jev para decisões estruturadas. **Status:** Definido.

**Decisão:** colocar o Jev no pipeline (entre Silver e Gold)

**Motivo:** é o ponto do pipeline onde há dado limpo o suficiente para uma decisão útil, mas ainda antes da modelagem final da Gold. **Alternativas consideradas:** nenhuma outra posição no pipeline foi discutida. **Status:** Definido.

**Decisão:** manter Bend2 isolado do pipeline

**Motivo:** Bend2 não tem integração com Spark/Databricks; forçar essa integração consumiria tempo sem gerar valor. **Alternativas consideradas:** usar Bend2 como linguagem do pipeline inteiro — explicitamente descartada (ver Seção 22). **Status:** Definido.

**Decisão:** não usar Bend2 como linguagem do pipeline

**Motivo:** Bend2 compila para JS/C/binário nativo, sem suporte a I/O distribuído, orquestração ou processamento de DataFrames Spark. O runtime de Bronze/Silver/Gold precisa rodar na JVM/Spark do Databricks. **Alternativas consideradas:** essa foi a pergunta original do autor ("podemos usar o Bend2 como linguagem de programação para fazer o pipeline?"). Rejeitada explicitamente. **Status:** Definido — decisão final e explícita da conversa.

**Decisão:** domínio de recuperação de crédito

**Motivo:** domínio real de trabalho do autor na Agecob; permite defender decisões de modelagem com conhecimento genuíno. **Alternativas consideradas:** nenhum outro domínio foi discutido. **Status:** Definido.

**Decisão:** priorizar o pipeline Databricks como "must-have" e tratar Jev/Bend2 como componentes menores em relação ao core

**Motivo:** risco de terminar com 3 coisas pela metade em vez de 1 coisa completa; um pipeline Medallion 100% completo vale mais no portfólio do que 4 peças inacabadas. **Alternativas consideradas:** tratar as 3 tecnologias com peso igual desde o início. **Status:** Recomendação dada pela IA; o autor optou por manter os três juntos desde o início ("os 3 juntos, a ideia é essa bagunça mesmo") — **decisão final do autor prevalece sobre a recomendação.**

## 22. O que foi Descartado

| Ideia                                                                                         | Motivo da rejeição                                                                                                                                                                 | Decisão atual                                                                                                                  |
|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Usar Bend2 como linguagem de implementação do pipeline inteiro                                | Bend2 não tem integração com Spark/Databricks; compila para JS/C/binário, sem suporte a I/O distribuído ou DataFrames; tentar forçar consumiria tempo sem gerar valor de portfólio | Bend2 usado apenas isoladamente, para prova formal de 1 regra de negócio (Seção 13)                                            |
| Focar apenas em Databricks hoje, deixando Jev e Bend2 para outro dia                          | Recomendação inicial da IA, dado o risco de três tecnologias novas simultaneamente                                                                                                 | O autor optou explicitamente por manter os três juntos ("a ideia é essa bagunça mesmo")                                        |
| Tratar Databricks, Jev e Bend2 com peso igual de esforço                                      | Risco identificado de terminar com três peças inacabadas em vez de uma completa                                                                                                    | Recomendação de priorizar o pipeline como "must-have" foi dada, mas a decisão final do autor foi manter os três — ver Seção 21 |
| Interpretar erroneamente o nome do projeto como indicando um domínio diferente (música/banda) | "Hail to the King" é nome de álbum do Avenged Sevenfold; a IA verificou com o autor antes de prosseguir e confirmou tratar-se apenas de um codinome                                | Nome mantido como codinome do projeto, sem relação com o domínio de negócio                                                    |

## 23. Pontos em Aberto

### Arquitetura

| Questão                                           | Por que importa                      | Dependência                 | Prioridade | Status |
|---------------------------------------------------|--------------------------------------|-----------------------------|------------|--------|
| Modelo dimensional exato da Gold (fato/dimensões) | Necessário para implementar a camada | Schema do dataset (Seção 8) | Alta       | Aberto |

| Questão                                   | Por que importa               | Dependência       | Prioridade | Status |
|-------------------------------------------|-------------------------------|-------------------|------------|--------|
| Mecanismo exato de idempotência na Bronze | Requisito crítico do pipeline | Schema do dataset | Alta       | Aberto |
| Mecanismo exato de incremental load       | Requisito crítico do pipeline | Schema do dataset | Alta       | Aberto |

## Dados

| Questão                                                      | Por que importa                                                       | Dependência     | Prioridade | Status |
|--------------------------------------------------------------|-----------------------------------------------------------------------|-----------------|------------|--------|
| Schema final do dataset sintético (tabelas, colunas, chaves) | Base de tudo o resto do projeto                                       | —               | Alta       | Aberto |
| Volume final de linhas geradas                               | Afeta desempenho e realismo do teste                                  | Schema definido | Média      | Aberto |
| Se haverá tabela de histórico de contato                     | Necessária para o indicador de custo-efetividade por canal (Seção 15) | Schema definido | Média      | Aberto |

## Databricks

| Questão                                     | Por que importa                                  | Dependência | Prioridade | Status                                                 |
|---------------------------------------------|--------------------------------------------------|-------------|------------|--------------------------------------------------------|
| Configuração de Storage/Lakehouse detalhada | Fase 0, item pendente                            | —           | Média      | Aberto                                                 |
| Configuração de Notebooks e Jobs            | Fase 0, item pendente                            | —           | Média      | Aberto                                                 |
| Secret scope para credenciais do Jev        | Necessário antes de integrar o Jev com segurança | —           | Alta       | Aberto — opção "Create a secret" já identificada na UI |

| Questão                                       | Por que importa       | Dependência | Prioridade | Status |
|-----------------------------------------------|-----------------------|-------------|------------|--------|
| Bibliotecas e dependências com versões exatas | Fase 0, item pendente | —           | Baixa      | Aberto |

## Jev

| Questão                           | Por que importa                       | Dependência               | Prioridade | Status                                   |
|-----------------------------------|---------------------------------------|---------------------------|------------|------------------------------------------|
| Endpoint exato da API             | Necessário para implementar a chamada | —                         | Alta       | Aberto — acesso confirmado, detalhes não |
| Método de autenticação            | Idem                                  | —                         | Alta       | Aberto                                   |
| Formato exato de request/response | Idem                                  | —                         | Alta       | Aberto                                   |
| Rate limits                       | Afeta design do step de decisão       | —                         | Média      | Aberto                                   |
| Teste mínimo de chamada real      | Valida que a integração é possível    | Endpoint/auth confirmados | Alta       | Aberto                                   |

## Bend2

| Questão                                        | Por que importa             | Dependência                        | Prioridade | Status |
|------------------------------------------------|-----------------------------|------------------------------------|------------|--------|
| Regra de negócio final a ser provada           | Define o escopo da Seção 13 | Conhecimento do domínio de acordos | Alta       | Aberto |
| Instalação local do Bend2 (via Bun) confirmada | Fase 0, item pendente       | —                                  | Média      | Aberto |

| Questão                                   | Por que importa                | Dependência     | Prioridade | Status |
|-------------------------------------------|--------------------------------|-----------------|------------|--------|
| Estrutura exata do mini-repositório Bend2 | Organização do código de prova | Regra escolhida | Baixa      | Aberto |

## Métricas

| Questão                                                          | Por que importa                      | Dependência       | Prioridade | Status |
|------------------------------------------------------------------|--------------------------------------|-------------------|------------|--------|
| Granularidade temporal da taxa de recuperação (mensal? semanal?) | Define a query/agregação             | Schema do dataset | Média      | Aberto |
| Viabilidade do indicador de custo-efetividade por canal          | Depende de tabela de contato existir | Schema do dataset | Média      | Aberto |

## Testes

| Questão                                              | Por que importa                              | Dependência           | Prioridade | Status |
|------------------------------------------------------|----------------------------------------------|-----------------------|------------|--------|
| Estrutura detalhada de cada tipo de teste (Seção 20) | Necessária para a Fase 7                     | Pipeline implementado | Média      | Aberto |
| Uso confirmado de pytest                             | Mencionado apenas como referência de mercado | —                     | Baixa      | Aberto |

## Infraestrutura

| Questão                                            | Por que importa                                              | Dependência | Prioridade | Status |
|----------------------------------------------------|--------------------------------------------------------------|-------------|------------|--------|
| Papel efetivo do Azure no projeto (ADF, ADLS Gen2) | Afeta se haverá camada de orquestração externa ao Databricks | —           | Média      | Aberto |

| Questão                                                       | Por que importa       | Dependência | Prioridade | Status |
|---------------------------------------------------------------|-----------------------|-------------|------------|--------|
| Configuração do repositório Git (branch, .gitignore, secrets) | Fase 0, item pendente | —           | Média      | Aberto |

Documentação

| Questão                                      | Por que importa                     | Dependência | Prioridade | Status |
|----------------------------------------------|-------------------------------------|-------------|------------|--------|
| Estrutura final de diretórios do repositório | Só uma sugestão foi dada (Seção 17) | —           | Baixa      | Aberto |

Portfólio

| Questão                        | Por que importa                                  | Dependência          | Prioridade           | Status |
|--------------------------------|--------------------------------------------------|----------------------|----------------------|--------|
| README final ainda não escrito | Só o que ele deve conter foi definido (Seção 26) | Projeto implementado | Baixa (por enquanto) | Aberto |

24. Riscos e Limitações

**Tecnologias muito novas (Jev e Bend2):** ambas lançadas em setembro de 2026, dias antes desta conversa. Documentação, preços e comportamento de API podem mudar rapidamente. **Fato conhecido**, não hipotético.

**API do Jev em early access/waitlist:** havia risco real de o autor não ter acesso liberado; o autor confirmou ter acesso, mas os detalhes técnicos da API (endpoint, formato) ainda não foram validados nesta conversa. **Risco parcialmente mitigado, mas não eliminado.**

**Mudanças recentes no ecossistema Databricks:** a Community Edition foi aposentada em 01/01/2026 e substituída pela Free Edition; a DP-203 foi aposentada em 31/03/2025 e substituída pela DP-700; Databricks renomeou "Asset Bundles" para "Declarative Automation Bundles". **Fato conhecido** — reforça a necessidade de verificar páginas oficiais antes de investir tempo/dinheiro.

**Limitações do ambiente gratuito (Databricks Free Edition):** serverless-only, sem compute customizado ou GPU, SQL warehouse limitado a 2X-Small, até 5 tarefas de job concorrentes, 1 pipeline Lakeflow ativo por tipo, sem R/Scala em notebooks, 1 workspace/metastore por conta, sem SLA/suporte garantido. **Fato conhecido, confirmado na prática durante o setup.**

**Dados sintéticos:** por serem gerados artificialmente, não capturam padrões reais de inadimplência; qualquer métrica calculada tem validade limitada como demonstração técnica, não como insight de negócio real. **Fato conhecido.**

**Faker não é motor de big data:** ótimo para prototipagem e volumes de milhares a poucos milhões de linhas; para dezenas de milhões precisa de otimização (NumPy, escrita em streaming). **Fato conhecido,** relevante se o volume final for maior do que o inicialmente estimado.

**Risco de overengineering:** empilhar Databricks + Jev + Bend2 num prazo curto pode resultar em três componentes mal implementados em vez de um pipeline sólido. **Risco identificado explicitamente na conversa,** mitigado parcialmente ao tratar Bend2 como peça isolada (não bloqueante do core) e ao definir critérios de sucesso claros para o pipeline.

**Risco de integrar ferramentas apenas por estética/modismo:** usar Jev e Bend2 "porque são novos" sem justificativa arquitetural sólida enfraqueceria o projeto em entrevista. **Risco identificado explicitamente** — mitigado ao definir papéis claros e justificativas para cada tecnologia (Seções 6 e 21).

## 25. Como Defender o Projeto em Entrevista

### Pitch de 30 segundos

"Construí um pipeline de dados completo no Databricks, usando arquitetura Medallion, para simular um ambiente de recuperação de crédito — o mesmo domínio onde trabalho hoje. Os dados brutos passam por limpeza e validação até virar um modelo dimensional pronto pra análise, com indicadores como taxa de recuperação e aging de carteira. No meio do caminho, uso um modelo de IA especializado em decisões estruturadas para priorizar quem contatar primeiro, e provo formalmente que uma regra crítica de negócio nunca gera inconsistência financeira."

## Pitch de 2 minutos

Expandir o pitch de 30s cobrindo: (1) por que Medallion (separação de responsabilidades, idempotência, incrementalidade, SCD Type 2), (2) por que o domínio de crédito foi escolhido (conhecimento real de negócio), (3) por que o Jev entra como decisão estruturada e não como chatbot (velocidade, saída tipada, sem risco de alucinação), (4) por que o Bend2 fica isolado do pipeline (limitação técnica real, não improvisado), e (5) os indicadores de negócio que o projeto entrega.

## Perguntas técnicas prováveis

| # | Pergunta                                                           | Resposta esperada                                                                                                           | Conceito avaliado     |
|---|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------|
| 1 | Por que separar em Bronze/Silver/Gold em vez de fazer tudo direto? | Isolamento de responsabilidades, permite reprocessar/auditar, serve diferentes consumidores sem acoplar lógica              | Medallion             |
| 2 | Como você garante que Spark processa isso de forma distribuída?    | Spark particiona os dados e processa em paralelo entre workers; no caso deste projeto, via compute serverless do Databricks | Spark                 |
| 3 | Por que Delta Lake e não Parquet puro?                             | Delta adiciona transações ACID, MERGE, time travel e schema enforcement sobre o Parquet                                     | Delta                 |
| 4 | Como você garante idempotência na ingestão?                        | Chave única + MERGE (upsert) em vez de INSERT simples                                                                       | Idempotência          |
| 5 | Como funciona seu carregamento incremental?                        | Watermark/timestamp identifica o que é novo desde a última execução                                                         | Incremental load      |
| 6 | O que é SCD Type 2 e onde você usou?                               | Técnica que preserva histórico criando novas linhas versionadas em vez de sobrescrever; usada em 1 dimensão da Silver       | SCD2                  |
| 7 | Como você modelou sua camada Gold?                                 | 1 fato + 2-3 dimensões, seguindo modelagem dimensional clássica                                                             | Modelagem dimensional |

| #  | Pergunta                                                   | Resposta esperada                                                                                                                       | Conceito avaliado              |
|----|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| 8  | O que é o Jev e por que você não usou um LLM comum?        | System 1 model, não generativo, rápido (70-500ms), saída estruturada garantida sem risco de alucinação                                  | Jev                            |
| 9  | O que significa "System 1" nesse contexto?                 | Analogia a decisão rápida/instintiva (vs. raciocínio deliberado de um LLM generativo)                                                   | System 1 models                |
| 10 | Como você trata erros na chamada da API do Jev?            | Depende da implementação final — <b>NÃO DEFINIDO</b> ainda, marcar como área a desenvolver                                              | APIs                           |
| 11 | O que é Bend2 e por que ele não roda dentro do Spark?      | Linguagem de prova formal com dependent types; compila para JS/C/binário, sem integração com Spark; usado isoladamente                  | Bend2                          |
| 12 | O que é uma prova formal e como ela difere de um teste?    | Teste verifica casos específicos; prova formal garante matematicamente a propriedade para todo o domínio de entrada                     | Provas formais                 |
| 13 | Por que você escolheu o domínio de recuperação de crédito? | Domínio real de trabalho do autor, permite defender decisões de modelagem com conhecimento genuíno                                      | Decisão arquitetural           |
| 14 | Como você decidiu o que é fato e o que é dimensão?         | Fato = evento mensurável granular (pagamento); dimensão = atributo descritivo (cliente, tempo, canal)                                   | Modelagem dimensional          |
| 15 | Se você tivesse dado real da empresa, o que mudaria?       | Volume real, distribuições reais de inadimplência, necessidade de anonimização/compliance — dados sintéticos não capturam esses padrões | Limitações de dados sintéticos |

As respostas de itens marcados **NÃO DEFINIDO** devem ser completadas conforme as decisões forem tomadas durante o desenvolvimento (ver Seção 23).

## 26. Como o Projeto Deve Aparecer no Portfólio

**Narrativa central:** "arquitetura de dados tradicional + decisão estruturada de IA + verificação formal" — um pipeline Medallion sólido, aplicado a um domínio real de negócio (recuperação de crédito), com dois diferenciais que mostram profundidade técnica sem virar modismo.

### O que deve aparecer no README:

- Visão geral do projeto e o problema de negócio que resolve (Seções 1-2)
- Diagrama de arquitetura (Seção 5)
- Papel de cada tecnologia e por que foi escolhida (Seção 6)
- Indicadores de negócio calculados, com explicação de cada métrica (Seção 15)
- Explicação clara de que o Bend2 é isolado do pipeline e por quê (Seção 13/21)
- Como rodar o projeto localmente/no Databricks

### Quais diagramas devem existir:

- Diagrama de arquitetura geral (Bronze→Silver→Jev→Gold, com Bend2 isolado — Seção 5)
- Diagrama do modelo dimensional da Gold (fato + dimensões), quando definido

### Quais resultados devem ser mostrados:

- Prints ou outputs dos indicadores de negócio calculados (Seção 15)
- Exemplo de chamada e resposta do Jev (sem expor credenciais)
- Resultado da prova formal do Bend2

### Quais decisões são interessantes para um recrutador:

- A escolha deliberada do domínio de crédito (conhecimento real de negócio)
- A justificativa de por que o Jev é diferente de um LLM comum
- A decisão explícita de NÃO integrar o Bend2 ao runtime do pipeline (mostra maturidade arquitetural, não só empilhar tecnologia)

### Quais partes devem ser demonstradas:

- O pipeline funcionando ponta a ponta (Bronze→Silver→Gold)

- A chamada real à API do Jev
- A execução da prova no Bend2

### Quais partes são secundárias:

- Detalhes de configuração do Azure (se usado) — papel ainda não confirmado (Seção 23)
- Estrutura interna de pastas (importa menos que a clareza da arquitetura)

Esta seção define o que o README deve conter — o README final em si ainda não foi escrito.

## 27. Referências e Documentação

### Documentação oficial

| Nome                            | URL                                                                                                                                 | Para que serve                                                 | Quando consultar                                   |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|----------------------------------------------------|
| Databricks Free Edition         | <a href="https://databricks.com/learn/free-edition">databricks.com/learn/free-edition</a>                                           | Cadastro e documentação da edição gratuita usada no projeto    | Fase 0 (setup)                                     |
| Databricks Academy              | — (mencionado, URL não fornecida na conversa)                                                                                       | Data Engineer Learning Plan gratuito                           | Estudo complementar / preparação para certificação |
| Microsoft Learn — trilha DP-700 | — (mencionado, URL não fornecida)                                                                                                   | Trilha "Elevate your Microsoft Fabric data engineering skills" | Estudo complementar                                |
| TypeSafe AI — blog              | <a href="https://typesafe.ai/blog/introducing-system-one-models-and-jev">typesafe.ai/blog/introducing-system-one-models-and-jev</a> | Explicação oficial do que é o Jev e os System 1 Models         | Antes de integrar o Jev (Seção 12)                 |
| TypeSafe AI — docs              | <a href="https://docs.typesafe.ai/introduction">docs.typesafe.ai/introduction</a>                                                   | Documentação técnica da API do Jev                             | Fase 0.3 e Fase 4 (integração do Jev)              |

| Nome                  | URL                                                                               | Para que serve                   | Quando consultar                               |
|-----------------------|-----------------------------------------------------------------------------------|----------------------------------|------------------------------------------------|
| TypeSafe AI — console | <a href="https://console.typesafe.ai">console.typesafe.ai</a>                     | Onde o acesso à API é gerenciado | Fase 0.3                                       |
| Bend2 — site oficial  | <a href="https://bend2.dev/notes/what-is-bend2">bend2.dev/notes/what-is-bend2</a> | Explicação do que é o Bend2      | Antes de começar a Fase 6                      |
| Bend2 — GitHub        | <a href="https://github.com/bendlang/bend">github.com/bendlang/bend</a>           | Código-fonte, guia, exemplos     | Fase 0.4 (instalação) e Fase 6 (implementação) |

## GitHub — projetos de referência (portfólio ponta a ponta)

| Nome                                                                                                                                                                                                       | Para que serve                                                                                                                                                       | Quando consultar                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <a href="https://github.com/shubhamkapur/databricks-end-to-end-data-pipeline-with-medallion-architecture-project">shubhamkapur/databricks-end-to-end-data-pipeline-with-medallion-architecture-project</a> | Referência mais completa: retail, Medallion, incrementalidade, idempotência, Structured Streaming, DLT, Unity Catalog, SCD Type 1 e 2, modelagem dimensional na Gold | Fase 3 (projeto-âncora do pipeline)                      |
| <a href="https://github.com/rnetya/azure-databricks-healthcare-lakehouse">rnetya/azure-databricks-healthcare-lakehouse</a>                                                                                 | Referência de dados 100% sintéticos, Medallion, quarentena de dados, governança, orquestração e CI/CD com GitHub Actions                                             | Referência de estrutura de repositório profissional      |
| <a href="https://github.com/nenimiwebuke/azure-data-engineering-portfolio">nenimiwebuke/azure-data-engineering-portfolio</a>                                                                               | Referência de ADF + ADLS Gen2 + Databricks + PySpark + Delta + Power BI + Terraform + CI/CD, incremental via watermark + Delta MERGE                                 | Se Azure for confirmado como parte do projeto (Seção 23) |

| Nome                                                            | Para que serve                                                                     | Quando consultar                            |
|-----------------------------------------------------------------|------------------------------------------------------------------------------------|---------------------------------------------|
| thiwankarandeni/azure-adventureworks-project                    | Referência de projeto AdventureWorks ponta a ponta                                 | Referência geral                            |
| anshlambagit/Data_Engineer_Roadmap (Ansh Lamba, YouTube+GitHub) | Projetos AdventureWorks e Spotify ponta a ponta com CI/CD                          | Referência geral e CI/CD                    |
| RihabFekii/azure-databricks-end-to-end-project                  | Medallion com Unity Catalog, external locations, notebook de modelagem dimensional | Referência de modelagem dimensional na Gold |

## Fontes de dados sugeridas (guia original)

| Nome                                              | Para que serve                                                                                                                            |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| NYC Taxi & Limousine Commission                   | Dataset-padrão de portfólio (Parquet, volume real) — não é o dado usado neste projeto, mas foi cogitado como alternativa no guia original |
| Portal Brasileiro de Dados Abertos (dados.gov.br) | Dados abertos brasileiros (Banco Central, Saúde, Previdência)                                                                             |
| Kaggle                                            | Datasets diversos, limite de 20GB por dataset                                                                                             |
| Faker (Python)                                    | Biblioteca usada para gerar o dataset sintético deste projeto (Seção 8)                                                                   |

Nenhuma URL foi inventada acima — apenas as mencionadas explicitamente na conversa ou no material anexado. Como Jev e Bend2 são lançamentos muito recentes (setembro/2026), suas documentações podem ter mudado desde esta conversa — **verificar as páginas oficiais antes de confiar em qualquer detalhe técnico específico.**

## 28. Glossário

| Termo                                  | Definição curta                                                                                                                               | Contexto neste projeto                                                                                                                                 |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Medallion Architecture                 | Padrão de organização de dados em camadas progressivas de qualidade (Bronze → Silver → Gold)                                                  | Estrutura central do pipeline: catálogo <code>hail_to_the_king</code> com schemas <code>bronze</code> , <code>silver</code> , <code>gold</code>        |
| Bronze                                 | Camada de dados brutos, como chegaram da fonte, sem transformação                                                                             | Dados sintéticos gerados via Faker, ingeridos sem limpeza                                                                                              |
| Silver                                 | Camada de dados limpos, validados, deduplicados e normalizados                                                                                | Onde ocorrem as regras técnicas de qualidade de dados ( <b>NÃO DEFINIDO</b> em detalhe ainda)                                                          |
| Gold                                   | Camada de dados modelados para consumo — métricas de negócio, dimensões e fatos                                                               | Onde os indicadores de recuperação de crédito serão calculados (modelo dimensional é HIPÓTESE)                                                         |
| Delta Lake                             | Formato de tabela open-source sobre Parquet, com suporte a transações ACID, time travel e evolução de schema                                  | Formato de armazenamento padrão do Databricks; usado (presumivelmente) em todas as camadas — <b>NÃO DEFINIDO</b> se será usado explicitamente em todas |
| Unity Catalog                          | Sistema de governança de dados do Databricks, com namespace de três níveis (catálogo → schema → tabela)                                       | Catálogo <code>hail_to_the_king</code> já criado, com os três schemas do Medallion                                                                     |
| SCD Type 2 (Slowly Changing Dimension) | Técnica de modelagem dimensional que preserva o histórico de mudanças de um registro, criando novas linhas versionadas em vez de sobrescrever | Cogitada para dimensões da camada Gold (ex.: histórico de status de dívida) — <b>NÃO DEFINIDO</b> se será aplicada                                     |

| Termo                               | Definição curta                                                                                                                                                                                                       | Contexto neste projeto                                                                                                                                               |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Idempotência                        | Propriedade de uma operação que produz o mesmo resultado final não importa quantas vezes seja executada                                                                                                               | Requisito desejável para os jobs de ingestão/transformação — <b>NÃO DEFINIDO</b> como será garantida                                                                 |
| Incremental load                    | Estratégia de carregar apenas dados novos ou alterados desde a última execução, em vez de recarregar tudo                                                                                                             | Alternativa a full load para a ingestão Bronze — <b>NÃO DEFINIDO</b> qual abordagem será usada                                                                       |
| ETL / ELT                           | Extract-Transform-Load / Extract-Load-Transform — ordem em que dados são extraídos, transformados e carregados                                                                                                        | O projeto segue essencialmente um padrão ELT (dados carregados brutos no Bronze, depois transformados nas camadas seguintes)                                         |
| Structured output                   | Saída de um modelo restrita a um formato/schema definido, em vez de texto livre                                                                                                                                       | Característica central do Jev — todo output é tipado, não há geração de texto livre                                                                                  |
| System 1 (modelo)                   | Termo usado pela TypeSafe AI para descrever o Jev: um modelo de decisão rápido, não autoregressivo, análogo ao "pensamento rápido" (System 1) da psicologia cognitiva, em contraste com LLMs generativos ("System 2") | Jev é usado no projeto para tomar decisões estruturadas sobre casos de crédito (ex.: classificação, priorização) — schema de entrada/saída ainda <b>NÃO DEFINIDO</b> |
| Dependent types (tipos dependentes) | Sistema de tipos em que tipos podem depender de valores, permitindo expressar propriedades matemáticas precisas no próprio tipo                                                                                       | Fundamento teórico do Bend2, usado para checar formalmente uma regra de negócio                                                                                      |

| Termo                                | Definição curta                                                                                                                                    | Contexto neste projeto                                                                                                                |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Formal proof / law<br>(prova formal) | Demonstração matemática, verificada automaticamente pelo compilador, de que uma propriedade sempre é verdadeira para um programa                   | O que o Bend2 permite fazer via a construção <code>law</code> ; regra candidata a ser provada ainda é <b>HIPÓTESE / IDEIA</b>         |
| Faker                                | Biblioteca Python para geração de dados sintéticos (nomes, CPFs, datas, valores etc.)                                                              | Fonte dos dados sintéticos de crédito que alimentam a camada Bronze                                                                   |
| Jev                                  | Modelo de IA "System 1" da TypeSafe AI, usado via API, para decisões estruturadas não-generativas                                                  | Componente de inteligência do pipeline, atuando entre Silver e Gold (posição exata <b>NÃO DEFINIDA</b> )                              |
| Bend2                                | Linguagem de programação com tipos dependentes e provas formais ("laws"), da Higher Order Company                                                  | Componente isolado do projeto, usado para provar formalmente uma regra de negócio — <b>não integrado</b> ao pipeline Spark/Databricks |
| Databricks Free Edition              | Camada gratuita do Databricks, somente serverless, com limites de concorrência e recursos (substituiu a Community Edition, retirada em 01/01/2026) | Ambiente de execução usado neste projeto                                                                                              |
| Spark / PySpark                      | Motor de processamento distribuído de dados; PySpark é sua API em Python                                                                           | Motor de execução por trás dos notebooks Databricks (versão confirmada: 4.2.0)                                                        |
| Warehouse (SQL Warehouse)            | Recurso computacional do Databricks dedicado a consultas SQL                                                                                       | Compute configurado na Fase 0 (tamanho 2X-Small, limite do Free Edition)                                                              |

Este glossário cobre os termos técnicos usados ao longo do dossiê. Termos de negócio específicos (ex.: taxa de recuperação, aging da carteira) estão definidos na Seção 15 (Métricas e Indicadores de Negócio), não repetidos aqui.

## 29. Estado Atual do Projeto

Snapshot honesto do que existe de fato, feito em produtos reais, versus o que ainda é planejamento. Atualizar esta seção à medida que o projeto avança.

### Definido

- Nome/codinome do projeto: **Hail to the King** (escolha pessoal do autor, sem relação com o álbum do Avenged Sevenfold).
- Domínio de negócio: recuperação de crédito / cobrança (alinhado ao trabalho real do autor na Agecob, Goiânia).
- As três tecnologias centrais e seus papéis: Databricks/Medallion (engenharia de dados), Jev (decisão estruturada via API), Bend2 (prova formal isolada de uma regra de negócio).
- Decisão arquitetural: Bend2 **não** será usado como runtime do pipeline (sem integração com Spark) — permanece um componente isolado, conectado apenas conceitualmente ao domínio do projeto.
- Workspace Databricks criado (Free Edition, região **us-east-2**), cadastrado com e-mail pessoal (gmail), não com e-mail institucional (PUC-GO, que apresentou problema de conexão).
- Compute configurado: SQL Warehouse serverless, tamanho **2X-Small** (limite do Free Edition).
- Versão do Spark confirmada em notebook: **4.2.0**.
- Unity Catalog criado: catálogo `hail_to_the_king`.
- Schemas criados dentro do catálogo: `bronze`, `silver`, `gold` (além dos schemas padrão `default` e `information_schema`).
- Objetivos técnicos e de negócio formulados (Seção 3), incluindo métricas de negócio reformuladas como KPIs de cobrança (Seção 15).
- Este dossiê mestre existe e cobre as 29 seções especificadas.

## Em implementação

- Fase 0 (Preparação e Configuração do Ambiente) — **parcialmente concluída**: workspace, compute e catálogo/schemas prontos; faltam secrets, teste de acesso à API do Jev, instalação local do Bend2, repositório Git formalizado e smoke tests (ver Seção 19 — Checklist).

## Pendente

- Schema exato do dataset sintético (entidades, colunas, tipos, volumetria) — Seção 8.
- Estratégia de ingestão Bronze (idempotência, incremental vs. full load) — Seção 9.
- Regras de limpeza/normalização/deduplicação da camada Silver, e se SCD Type 2 será aplicado — Seção 10.
- Modelo dimensional da camada Gold (fato/dimensões) — ainda HIPÓTESE — Seção 11.
- Detalhes de integração do Jev: schema de input/output, tratamento de erros, limitações reais da API — Seção 12.
- Regra de negócio candidata à prova formal em Bend2, e definição de `type / def / law` — Seção 13.
- Plano de testes e validação (Seção 20) — nenhum item definido em detalhe ainda.
- Estrutura definitiva do repositório Git (Seção 17 apresenta apenas uma estrutura recomendada).
- README e materiais de portfólio (Seção 26).
- Confirmação de preço/limites atualizados do Jev (tecnologia muito recente, lançada em setembro de 2026).

## Bloqueado

- Nenhum bloqueio identificado no momento. O problema de cadastro com e-mail institucional (PUC-GO) foi contornado usando e-mail pessoal, não representando um bloqueio ativo.

## Próxima ação recomendada

1. Concluir a Fase 0: configurar secrets no Databricks, testar a chamada à API do Jev, instalar o Bend2 localmente (via Bun) e formalizar o repositório Git com a estrutura da Seção 17.
2. Definir o schema do dataset sintético (Seção 8) e implementar a geração via Faker.

3. Implementar a ingestão Bronze de acordo com a Fase 1 do Plano de Implementação (Seção 18).
4. Usar a Seção 19 (Checklist) como guia de progresso, marcando itens à medida que forem concluídos.

Este dossiê foi elaborado para ser consultado de forma independente, sem necessidade de retomar a conversa original. Sempre que uma pendência listada aqui ou na Seção 23 for resolvida, esta seção deve ser atualizada para refletir o novo estado real do projeto.